

 hyckomatej Update materials

03c5a68 · 3 months ago 

Name		Name	Last commit date
	..		
	common-errors	Update materials	3 months ago
	test-examples	Update materials	3 months ago
	README.md	Add lesson 5 materials	11 months ago

README.md



5. Lecture - Best development practices and debugging

This week, we are taking a look at a few common errors you're likely to run into while developing Solana programs. Understanding these errors will help you debug more quickly and effectively.



Tip

Check out [examples](#)! Try running the code and resolving the errors yourself.

Table of Contents

- [Types of Tests](#)
 - [Units Tests](#)
 - [Integration Tests](#)
- [Common Errors](#)
 - [Signer Verification Failed](#)
 - [No Prior Credit](#)
 - [Account Already In Use](#)
 - [Account Did Not Deserialize](#)
 - [Constraint Seeds](#)
 - [Program Failed To Complete](#)
- [On-chain Data Fetching](#)

Types of Tests

You should always aim for high test coverage. This includes testing both the "happy path" (successful cases) and "unhappy path" (error cases).

Units Tests

Unit tests are small tests that focus on one individual component or function at a time. They isolate logic to verify that each part works as expected under different conditions.

Unit tests are useful for:

- Checking boundary conditions, such as maximum or minimum values.
- Verifying basic functionality of individual instructions.

Here is an example of a simple unit test:

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_math_function() {
        // Check that the function returns `Some(8)` when input is within bounds
        assert_eq!(math_function(2), Some(8));
        // Check that the function returns `None` when input exceeds bounds
        assert_eq!(math_function(11), None);
    }
}
```



Integration Tests

Integration tests check whether different parts of a program work together as expected. They simulate real-world scenarios by testing more complex transactions and interactions between accounts.

Integration tests are useful for:

- Confirming program behavior across multiple accounts and instructions.
- Testing interactions with external programs or system accounts.

Here is an example of an integration test:

```
it("Cannot initialize with incorrect data account!", async () => {
  const bad_data = Keypair.generate();

  try {
    // Attempt to initialize the program with the incorrect account.
    await program.methods
      .initialize(10)
      .accountsStrict({
        user: user.publicKey,
        data: bad_data.publicKey,
        systemProgram: SystemProgram.programId,
      })
      .signers([user])
      .rpc();
  } catch (_err) {
    // Make sure that the error matches the expected "ConstraintSeeds" error.
    const err = anchor.AnchorError.parse(_err.logs);
    assert.strictEqual(err.error.errorCode.code, "ConstraintSeeds");
  }
});
```



On-chain Data Fetching

After sending a transaction during testing, it's essential to fetch and verify on-chain data to ensure expected changes were made.

Use the `fetch` function to retrieve account data:

```
// Fetch and verify the on-chain data.  
let dataAccount = await program.account.myData.fetch(data);  
assert.deepEqual(dataAccount.authority, user.publicKey);  
assert.strictEqual(dataAccount.counter, 0);
```



Common Errors

Signer Verification Failed

Cause

This error occurs when a required signer has not signed the transaction, which prevents the program from verifying authority for that action.

Example

In this example, `user` is set as payer for initializing the `data` account. However, this transaction will fail because each account has to also sign its initialization.

```
it("Is initialized!", async () => {
  const tx = await program.methods
    .initialize()
    .accountsStrict({
      user: user.publicKey,
      data: data.publicKey,
      systemProgram: SystemProgram.programId,
    })
    .signers([user]) // Missing signer.
    .rpc();
});
```



Solution

If an account is a PDA, we do not include it as a signer because the program will sign on its behalf. In this case, both `data` and `user` are regular accounts, so they both need to be included as signers.

```
.signers([user, data])
```



No Prior Credit

Cause

This error typically occurs when a transaction requires the payer account to have enough SOL to cover the cost of the transaction, but it has insufficient balance.

Example

In this example, we are again calling the `initialize` instruction again. However, this time there is nothing wrong with the code itself. The problem is that we did not ensure that accounts used for testing had sufficient balance to cover the transaction costs.

```
it("Is initialized!", async () => {  
  const tx = await program.methods  
    .initialize()  
    .accountsStrict({  
      user: user.publicKey,  
      data: data.publicKey,  
      systemProgram: SystemProgram.programId,  
    })  
    .signers([user, data])  
    .rpc();  
});
```



Solution

Always make sure to airdrop SOL to the accounts used for testing before running the tests.

```
before("prepare", async () => {  
  await airdrop(connection, user.publicKey);  
});
```



Here is an example `airdrop` function you can use:

```
async function airdrop(connection: any, address: any, amount = 1000000000) {  
  await connection.confirmTransaction(  
    await connection.getLatestBlockhash(),  
    connection.feePayer.publicKey,  
    connection.commitment,  
    connection.blockhash,
```



```
    await connection.requestAirdrop(address, amount),  
    "confirmed"  
  );  
}
```

You can retrieve account balance using the `getBalance` function:

```
await connection.getBalance(user.publicKey)
```



Account Already In Use

Cause

This error occurs when trying to initialize an account that already exists.

Example

In this example, we are trying to call `initialize` instruction twice with the same `data` account.

```
it("Is initialized!", async () => {  
  const tx = await program.methods  
    .initialize()  
    .accountsStrict({  
      user: user.publicKey,  
      data: data.publicKey,  
      systemProgram: SystemProgram.programId,  
    })  
    .signers([user, data])  
    .rpc();  
})
```




```
const repeat_tx = await program.methods
  .initialize()
  .accountsStrict({
    user: user.publicKey,
    data: data.publicKey,
    systemProgram: SystemProgram.programId,
  })
  .signers([user, data])
  .rpc();
});
```

Solution

Whenever this error arises, make sure to check that the correct accounts are being passed in the transaction. In our example, simply use a different account for the second `initialize` call.

```
const repeat_tx = await program.methods
  .initialize()
  .accountsStrict({
    user: user.publicKey,
    data: data2.publicKey,
    systemProgram: SystemProgram.programId,
  })
  .signers([user, data2])
  .rpc();
```



Account Did Not Deserialize

Cause

This error occurs when a program cannot interpret the data in an account according to its expected struct format.

Example

In this example, we are working with the context of `initialize` instruction, which initializes `data` account of type `MyData`. However, the allocated space does not include the 8 bytes required for the account discriminator, which is needed for deserialization.

```
#[derive(Accounts)]
pub struct Initialize<'info> {
    #[account(mut)]
    user: Signer<'info>,
    #[account(init,
        space = 32 + 1,
        payer = user,
    )]
    data: Account<'info, MyData>,
    system_program: Program<'info, System>,
}
```



```
#[account]
pub struct MyData {
    authority: Pubkey,
    counter: u8,
}
```



Solution

Deserialization issues often result from mismatched data formats or incorrect space allocation. To fix this, allocate 8 more bytes for the account discriminator.

```
#[account(init,  
    space = 8 + 32 + 1,  
    payer = user,  
)]  
data: Account<'info, MyData>,
```



Constraint Seeds

Cause

This error occurs when the seeds provided to derive a PDA do not match those specified in the program. It's crucial to keep the seed **order** and **values** consistent.

Example

In this example, we have defined `data` as a PDA in the program:

```
#[account(init,  
    space = 8 + 32 + 1,  
    payer = user,  
    seeds = [b"data1", b"data2"],  
    bump  
)]  
data: Account<'info, MyData>,
```



Now, let's derive the PDA for `data` in our test to pass it to a transaction. However, neither `option1` nor `option2` will work as expected:

`option1` uses incorrect values:

```
const option1 = PublicKey.findProgramAddressSync(  
  [Buffer.from("data"), Buffer.from("data2")],  
  program.programId  
)[0];
```



`option2` uses correct seed values but in the wrong order:

```
const option2 = PublicKey.findProgramAddressSync(  
  [Buffer.from("data2"), Buffer.from("data1")],  
  program.programId  
)[0];
```



Solution

To fix this error, make sure that both seed **order** and **values** match exactly what is specified in the program.

```
const correctPda = PublicKey.findProgramAddressSync(  
  [Buffer.from("data1"), Buffer.from("data2")],  
  program.programId  
)[0];
```



Program Failed To Complete

Cause

This error can happen when the program encounters a panic or an unhandled condition, causing the transaction to fail.

Example

In this example, we subtract two numbers inside `initialize` instruction:

```
pub fn initialize(ctx: Context<Initialize>, count: u8) -> Result<()> {  
    let data = &mut ctx.accounts.data;  
  
    data.authority = ctx.accounts.user.key();  
    data.counter = 10 - count;  
  
    Ok(())  
}
```



We pass a value for count that is intentionally too high so that the subtraction underflows and transaction fails.

```
it("Is initialized!", async () => {  
    const tx = await program.methods  
        .initialize(11) // The value 11 is intentionally too high.  
        .accountsStrict({  
            user: user.publicKey,  
            data: data,  
            systemProgram: SystemProgram.programId,  
        })  
        .signers([user])
```



```
.rpc();  
});
```

Solution

This time we are not really fixing the error, but we can make the program more resilient with custom error handling and clear error messages.

```
// Prevent underflow by checking that `count` does not exceed 10.  
require!(count <= 10, MyError::InvalidInstructionData);
```



```
#[error_code]  
pub enum MyError {  
    // Add custom error for clearer error messages.  
    #[msg("Invalid instruction data")]  
    InvalidInstructionData,  
}
```



Now the program will return a meaningful error if count is too high, preventing unexpected panics and making debugging easier.

Need help?

If you have any questions feel free to reach out to us on [Discord](#).

